Niagara 4 point deployment · Part 2 of 2

Collecting Those Points

The station as a client: devices, points, proxy extensions, proof

Verified on Niagara 4.14.0.162 · every reading driven from outside the station

Part 1 covers the other direction: publishing station points as BACnet, oBIX, Modbus slave

Shanghai Gline Net Co., Ltd.

@jason912 · jason.zhang@gline-net.com

Four steps, and the third one is silent when wrong

Network device point with a proxy extension proof from outside

Step	Object to create	Fails when
1 · Network	driver network of the client flavour, enabled	it is left disabled every child is inert
2 · Device	one object per field controller (address, port, poll)	address or port is wrong points go {stale}
3 · Point	point + proxy extension bound to one address	the extension is missing 0.00 {fault,stale} forever
4 · Proof	external read/write on the far end	nothing this is the step that cannot be faked

A point without its proxy extension is still a point

add creates the component, not the binding. The object shows a value slot, a status slot and a name, and reports 0.00 {fault,stale} with an empty faultCause the same appearance as a point that is polled and decoded wrongly. Only the extension tells you which of the two you have.

cp from a point that already carries the right extension is the working shortcut, but it copies the extension's settings too dataType, dataAddress and byteOrder come along and must be re-checked on every copied point.

Building the chain over the API

Four calls per protocol and the one field that decides whether it will ever poll

Step	Operation	What goes wrong without it
1 · Network	add + enabled = true	a disabled parent makes every device and point inert
2 · Device	add, then ipAddress / port / deviceAddress	wrong port or unit id every point {stale} with no fault
3 · Point	add with PROXY = <module>:<ProxyExt>	no PROXY the point keeps a NullProxyExt and never polls
4 · Proxy slots	oBIX PUT for dataType and dataAddress	both are enum or structured slots; mod rejects them
5 · Restart	once, before believing any reading	an assembled proxy reads 0.00 {fault,stale} until the station restarts

The point call in full

```
{"OP":"add","PATH":"/Drivers/ModbusTcpNetwork/<device>/points",
 "N":"<point>","T":"control:NumericPoint",
 "PROXY":"modbusCore:ModbusClientNumericProxyExt"}
```

proxyExt is a protected slot: it cannot be added after the fact, removed, or given a child. PROXY at creation is the only route.

cp from a point that already carries the right extension also works but it brings dataType, dataAddress and byteOrder with it.

The client chain per protocol

Same four steps, four different object names

	Modbus TCP	BACnet/IP	SNMP	oBIX
Network	modbusTcp:ModbusTcpNetwork	bacnet:BacnetNetwork	nSnmp:SnmpNetwork	obix:ObixNetwork
Device	ModbusTcpDevice	BacnetDevice	SnmpDevice	oBIX device
Address	ipAddress, port,deviceAddress	address(BacnetAddress)	ipAddress, port,community	URL path
Proxy ext	ModbusClientNumericProxyExt	BacnetNumericProxyExt	SnmpNumericProxyExt	ObixNumericProxyExt
Polled by	devicePollConfig+ pollScheduler	pollFrequency,COV	pollScheduler	poll rate
Outside check	FC03 / FC06	ReadProperty	snmpget	oBIX GET

Two traps in the table

nSnmp:SnmpNetwork is the nDriver type; the legacy snmp:SnmpNetwork still exists and behaves differently. One SNMP network per station.

BACnet collects on COV as well as polling, so an empty normalCount on a BACnet network is not automatically a fault check COV before judging the scheduler.

Four signals that report success without collection

All four are readable over REST, and none of them is a measurement

Signal	What it actually proves
device status = {ok}	the configuration was accepted by the driver
health = Ok [timestamp]	a PingMonitor read succeeded, on a separate path
readValue = 0.00 {ok}	a value object exists; the status facet is ok
readStatus = okNotActive	a poll range exists and nothing is scheduled

And one appearance that means two opposite things

```
out       = 0.00 {stale}
faultCause = ""          # empty     no fault, no hint
enabled    = true
```

A point polled and decoded wrongly looks exactly like a point never polled. The counters separate them; the value alone never will.

A clone of a healthy device is the worst case: it inherits values, an open socket and a climbing poll count, and has collected nothing.

Two counters settle it

Read the scheduler twice, ten seconds apart

pollScheduler did any poll fire at all?

/drivers/<driver network>/pollScheduler

Observation	Conclusion
totalPolls increases	the scheduler runs; look downstream (decode, mapping, device reply)
totalPolls frozen, normalCount = 0	no point was ever registered the queue is empty

Two networks side by side make it unmistakable

Network	Scheduler class	totalPolls	normalCount
SnmpNetwork1	NPollScheduler	22k, still climbing	current = 1
ModbusTcpNetwork	BasicPollScheduler	12k, frozen at 28 s	current = 0

Read both networks twice. A frozen totalPolls on a scheduler that started hours earlier means the queue stopped; the neighbouring network proves the station's schedulers still run. devicePollConfig describes intent, not outcome a range with consecutivePointsToPoll = 0 is not automatically the root cause, and setting it to a non-zero value produces no extra polls by itself.

The only proof: change the value from outside

Configure nothing, trust nothing, drive the far end

1. **Read the far end independently**
   Modbus master, snmpget, BACnet ReadProperty from another host

2. **Write a distinctive value at the source**
   a number you would recognise, or a quantity that moves

3. **Watch the station point**
   it must follow to the new value with {ok}

Worked loop: one slave, one client point, same station

```
FC06 write HR[1] = 7777      slave point Srv_Reg1 = 7777.0 {ok}
FC03 read  HR[0..1]          04d2 1e61   (register 1 = 7777)
client point C1              6666.0 {stale}      did not follow
```

A write that lands at the far end and does not appear in the client point is the failure this whole part is about and it is invisible from every status slot.

The same station's SNMP scheduler kept counting while the Modbus one was frozen, so the station was not idle: that one point's queue was.

Diagnostic order

Each step removes a whole class of causes, and the first four are side-effect free

#	Check	If it fails
1	Is the driver network enabled and {ok}?	enable it a disabled parent makes every child inert
2	Is pollScheduler.totalPolls increasing?	frozen with normalCount = 0: no point is registered
3	Do the points appear in devicePollConfig with a sane range and dataType?	fix the range; verify proxyExt/dataType
4	Does the device health timestamp advance?	the far end does not answer the ping address
5	Change the value at the far end does the point follow?	this is the only failure that survives 1 4

Two readings that look contradictory and are not

health advancing while points stay stale: the far end answers the ping address but not the point addresses compare pingAddress against each dataAddress.

readStatus reaching 0: ok while points stay stale: the range was accepted by the driver and still produced no usable reply.

An open case, stated as open

Three devices stale beside one that collects under identical conditions

Three Modbus client devices pointed at the same gateway, on the same network, at the same moment as a working client device, held 29 30 points {stale} while that neighbour collected normally. Per-range readStatus reached 0: ok while the points stayed stale.

Ruled out by measurement

far-end reachability TCP open from the station, real values read from outside
addressing deviceAddress, base address, range and dataType identical to the working device
enabled state, range size, and a full station restart

What is safe to conclude

the driver works proven by the working device in the same second
the fault is specific to those three device objects
resolving it needs in-station inspection or reconstruction, not more REST probing

> The transferable lesson: a working neighbour is the most valuable diagnostic. When one device collects and another does not under identical conditions, the network, the driver module and the protocol are all exonerated in a single step.